CC3K

Terry Zhou, Michael Wu, Alex Merila

Introduction:

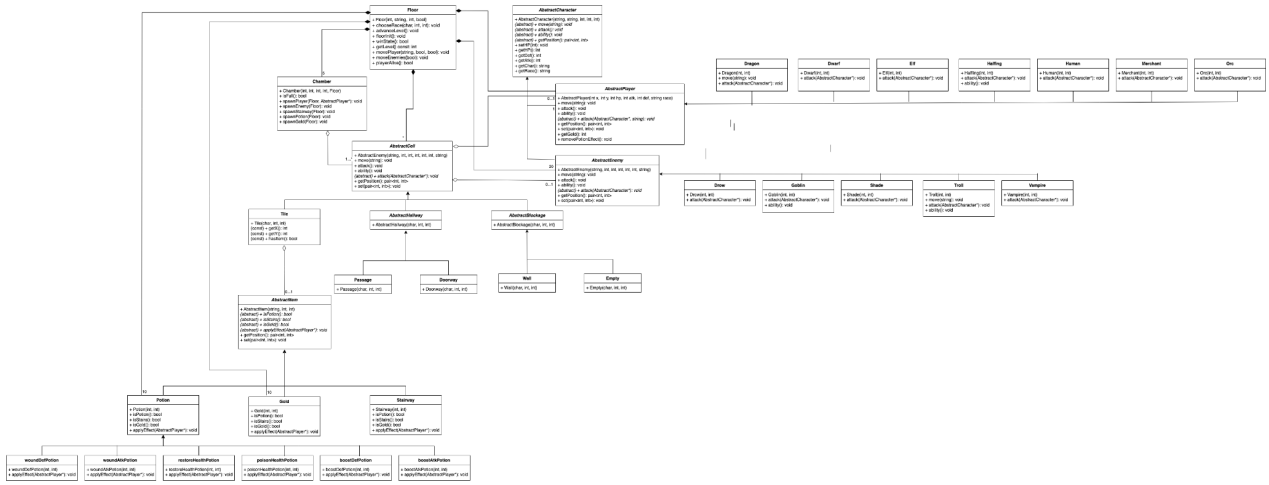
Though CC3K seemed like a simplistic Rogue clone in the beginning, we quickly found that even when armed with the power of C++, implementation soon proved to be non-trivial. This project provided a lot of insight into how we encapsulate the implementations of our code.

Overview:

Our rendition of Rogue is based on a few base class primitives from which all the other objects inherit from. They are the *AbstractCharacter, AbstractCell, AbstractItem,* and *Floor* classes. From these building blocks we construct the levels after parsing the provided empty level files and populate them with randomly generated treasures, potions, and enemies.

Updated UML class model:

Many of the changes in our UML structure came from having to deal with unexpected time constraints and implementation complexity. Specifically, we disregarded the decorator for simplicity. Instead, we made the potion effects into subclasses of the Potion class. We also give AbstractCells access to the Player and the Enemies for easier coding.



Design:

- Spawning and level generation
    - To spawn our game map, we decided to create a floor and read in every character of every line from an input file. As each char is processed, it is evaluated to become a certain object (i.e. '.' gets scanned, and a Tile object is created). These cells are filled within a vector list of AbstractCell to create the whole game map.
    - The chambers are formed by specifying coordinates/boundaries of the map from the top right to the bottom left of where the chambers should be. Each chamber is then fed all the abstractCells within those boundaries into a vector list of AbstractCell. The identification of these chambers would later be used to identify random generation of the character within the game map. Floor will keep track with a vector list of chambers and count of them.
    - The player is spawned by selecting a random chamber based on a count of them using rand() methods, and within the randomly selected chamber, it will select a random cell within the vector list of cells within the Chamber class, also using rand().
    - Similarly, the potions, gold, and enemies also select a random chamber and then random cells within the chamber. They are within a for loop that iterates up to a desired amount of randomly placed items and enemies.
    - Everything is formed using smart pointers, particularly shared or unique pointers in order to manage memory well.
- Player and enemy abstractions
    - The players and enemies are all derived from the *AbstractCharacter* base class, since they share operations such as movement and attacking. This allowed us to reuse code and add custom behavior such as abilities and races easily by extending this base class.
    - Since players and enemies share a common interface, it would be easy to add more complex AI behaviours and logic to enemies instead of just giving them randomized movement.
    - In turn, if the game provided different input methods such as a controller or touchscreen, the movement methods could easily be overloaded with such input whilst still providing the same behavior
- Items
    - We created three item subtypes: potions, gold, and the stairway. This design decision was prompted by the fact that all of these objects share an interactive element with respect to the player, that is they hold specific

effects that are triggered when the player comes into contact with them, such as changing the state of the player or the level.
    - One of the divergent changes from the initial UML was to no longer implement potions using the Decorator Pattern. This was due to a lack of understanding and foresight, as in the end it proved to make the effect reset upon level change much more complicated than it should have been. We instead opted out to overload a single *applyEffect()* method which accessed the fields (yes, fields- almost no getters/setters here!) of the player and directly mutated them.
    - Another design decision was to not create different objects for the piles of gold and instead add a value field for the different types of hordes. This proved more efficient in the simple format of a Roguelike, but if more complex graphics were required then separate objects would likely be required for more valuable treasures.
- Parsing input
    - We implemented many of the input parsing conditionals using switch case statements where possible, because they provide greater execution speed when compared to if-else statements. However, a drawback of this is that only characters or integers can be matched, and not strings, which could result in issues if the commands grow in complexity, and the fact that the speed is negligible in a game with ASCII graphics.
        - However, should the game have a graphical interface or even leverage a game engine, it could still rely on the ASCII level files which greatly facilitate level design. A prime example of this is the Roguelike Spelunky 2, which boasts expensive graphics but still uses ASCII level files.

Resilience to Change:

Coming into CC3K, our group had planned out, through our original UML, several design patterns in order to implement the classes of the project. Starting with the characters, we created an AbstractCharacter class in order to be the superclass to the players and enemies of the game. The AbstractCharacter class acted as the base to all characters of the game because all characters have HP, Atk, and Def values attached to them. AbstractPlayer class and AbstractEnemy classes were then created to inherit from the AbstractCharacter to differentiate from the player character which the user controls, and the enemies which the user does not control. With this distinction, subclasses of AbstractPlayer and AbstractEnemy can be formed, each with different HP, Atk, Def values, and

abilities. Our game can easily attach a new race of either player or enemy by creating a subclass that inherits from either AbstractPlayer or AbstractEnemy and override any methods that differ from race to race (i.e. their attacking abilities or movement).

Similarly, we created an AbstractItem class to be the superclass of all items, and had Potions and Gold inherit from AbstractItem.Potions have their own inherited subclasses based on the type of potion, particularly with pure virtual method applyEffect() that is overridden with each subclass of potion to fit its desired effect. We decided that a pure virtual method would be much more simpler and efficient to make it mandatory that any potion has an effect, rather than a Decorator pattern that is attached to the player and decorates the potion's effects. As for gold, we decided to have one central gold class that takes in a value when it is constructed, so gold can be any arbitrary value, making adjustments to how much gold a player can collect much easier to change (i.e. if we decided a normal stack is worth 3 instead of 2).

To put all our objects into one place, we created a Floor class that would contain all our objects. The Floor class would contain pointers of the Player, a vector list of enemies, a vector list of potions, vector list of gold, and the most importantly, a vector list of AbstractCells. The AbstractCells represent the objects of the game map, with each AbstractCell having subclasses like Walls, Entrances, Pathways, and Tiles. Each cell should have either player, enemy, or item sitting on it, similar to how a person would stand on a floor tile in real life, so we attached AbstractPlayer, AbstractEnemy, and AbstractItem within the field of an AbstractCell. This way, we could easily add, remove, and check if there is any object "on a" cell. Additionally, if we ever want to add a new feature (e.g. hidden items inside the wall), we can easily add an item, player, or enemy to be in any cell.

Many of our constructor's fields are not in public, leaving them protected from unauthorized change. The constructors of many of our objects on the other hand, can be easily modified based on the user's design. For instance, consider that most of our classes also have a displayChar that is formed in the superclass constructor. This can also be easily changed to any character and applied to every object or certain objects.

In the design of our spawning mechanisms we employed a Random Number Generator which used a seed based on the current time of the machine. However, in the scenario that we wish to add a multiplayer experience, we would

require that all the random generation is strictly deterministic so that all of our clients can generate the same levels locally and facilitate the workload of the server. Thus, in a hypothetical multiplayer version we would supply the same seed to all of the players so that we still maintain the accurate pseudo random generation whilst also providing consistent and reproducible level generation on all of the clients. Further, since the underlying movement and attack logic of the players and enemies is virtually the same, implementing a Player vs Player experience would be viable with light tweaking, especially if hosted on a local machine.

Another aspect of our code that is resilient to change is the option to alter the ASCII display character for many of the objects. Should the client want to use different characters or colors for the terminal display, they only need to supply their desired character to the object constructor. For example, if they wish potions to be represented using *p* and the purple color, they would only have to provide the specific ANSI escape code string with the custom character.

Questions:

What lessons did this project teach you about developing software in teams? If you worked alone, what lessons did you learn about writing large programs?

We learned that developing software in teams, no matter how small, can become an arduous task. One of the biggest problems we had to deal with was resolving Git merge conflicts: though we worked on separate branches and different features, any overlap resulted in conflicts that posed a challenge. We approached this by manually accepting incoming changes and tweaking code, or more crudely by cloning the working main branch and manually adding new code to it. In retrospect, better knowledge of Git merge mechanics would have been a great timesaver.

Another lesson we learned about developing in teams is that it is difficult to distribute labour between teammates and communicate properly. For example, when implementing the chambers found on each floor, we did not have a clear idea whether to hardcode their boundaries or create an algorithm to dynamically parse them. Further, minor miscommunication occurred and two teammates started working on implementing this feature independently. This was caused by unclear planning and distribution of tasks, since at the time we did not realize the

time requirements for each of the features and the complexity of the implementations.

One other valuable lesson we learned is with respect to meeting deadlines. In scenarios where features were not finished at the self-imposed deadlines, extra work had to be done by the whole team to compensate, including on hacking or hardcoding an implementation that violated the UML. Furthermore, finding the time to meet up in person to work on the project also proved difficult at certain points due to different schedules.

Moreover, a very valuable obstacle we had to deal with was creating readable code and using comments. Though we aimed for encapsulation so that the implementation would be abstracted away, when faced with having to read each other's code or use its interface, problems arose. As the implementation grew bigger, the codebase started to resemble the infamous spaghetti code as we all scrambled to implement the features in due time.

The last lesson we learned is that working in a team to create a project we are all invested in is that collaborating is a pleasant experience, as it facilitates meaningful discussion and exchange of ideas in a positive environment.

What would you have done differently if you had the chance to start over?

A lot of things. Chiefly, we would start implementing earlier so that we could face the biggest obstacles faster and reiterate on our design.

One of the most glaring issues in our codebase is the lack of design pattern usage. Due to time constraints, features such as enemy generation which was a perfect contender for the Factory Method, or the potions which should have been implemented using the Decorator Pattern, ended up being implemented with a disregard for these patterns.

Another feature which would have proven too complex to implement is parsing custom chamber layouts. The only hypothetical solution we found requires using a graph algorithm to recursively find the boundaries of each chamber. Thus, we resorted to hardcoding the coordinates of all the chamber boundaries.

Another practice we would try given another change at implementing is peer code review, as this would help with avoiding a lot of the most common pitfalls and issues that arose, especially when having trouble with working with code written by another.

Another very important factor we should have implemented is to use unique_ptrs more often than shared_ptrs, to help manage distinct ownership. Using unique_ptrs would have probably forced us to adapt to a more encapsulated design.

Conclusion:

This project has been a good exercise to train our consistency with Object Oriented Programming, as we manage the creation of objects, which objects has or owns what, inheritance between classes, among many other concepts. The project was also a great way to become accustomed to working in a group where there are many pieces that need to be put together in order to create the entire project (along with GitHub repository management within collaborators). To conclude, this project has provided a very valuable and fun learning experience.Thank you for reading!